Contents

Day 4 — Evals

The Evaluation Loop Is the Core of AI Engineering

If Day 2 established that **context is part of the program**, and Day 3 established that **retrieval is an information-retrieval system**, Day 4 introduces the mechanism that makes both of those systems engineerable:

evaluation.

This is arguably the most important day of Week 1.

Traditional software engineering gives us a powerful development loop:

```
Code
  ↓
Tests
   ↓
Failure
   ↓
Fix
   ↓
Tests
```

The tests provide a stable reference point.

Without them, changing the software becomes dangerous because there is no reliable way to determine whether the system improved or regressed.

AI applications require the same discipline.

The difference is that many of their outputs are not deterministic.

A traditional function might have a contract:

$$f(2, 3) = 5$$

An LLM application might instead produce several acceptable outputs:

```
"Revenue increased by 14.2%."
"Revenue grew 14.2% year over year."
"Revenue was up approximately 14%."
```

All three may be correct.

This means traditional exact-output testing is insufficient.

We need a richer concept:

> **An evaluation defines what good behavior means and measures how consistently the system achieves it.**

The fundamental development loop therefore becomes:

```
Application
     ↓
Evaluation
     ↓
Metrics
     ↓
Failure analysis
     ↓
Application change
     ↓
Evaluation
```

This is the beginning of **eval-driven development**.

---

1. Why AI Systems Need Evals

Consider an AI assistant that answers questions over company documentation.

Version 1 achieves:

$$\text{Accuracy} = 87\%$$

An engineer changes:

- the system prompt
- the retrieval algorithm
- the chunk size
- the model
- the context ordering

The new system feels better.

But what is the actual accuracy?

Perhaps:

$$\text{Accuracy} = 81\%$$

The engineer has accidentally introduced a regression.

Without an evaluation suite, this may go unnoticed.

This is especially dangerous because AI regressions are often subtle.

A change might:

- improve simple questions
- break complex questions

- improve factuality
- reduce completeness
- improve retrieval recall
- increase hallucination
- reduce latency
- increase cost
- improve average performance
- catastrophically degrade a rare but important class of requests

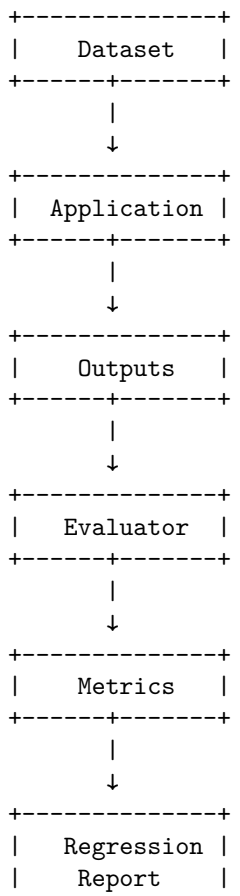
A single aggregate score cannot capture all of these dimensions.

This is why evaluation must be treated as a first-class engineering subsystem.

---

2. The Evaluation Harness

The basic architecture is:



```
              +--------------+
```

This should be treated as infrastructure.

The evaluation harness should be able to run against different versions of the application:

```
application_v1
application_v2
application_v3
```

and answer:

> What changed?

That is fundamentally different from manually trying the application and deciding whether it "seems better."

---

3. Golden Datasets

The foundation of offline evaluation is the **golden dataset**.

A golden dataset contains representative inputs together with expected behavior.

For a simple question-answering application:

```
{
    "input": "What is the maximum hotel reimbursement?",
    "expected_answer": "$350 per night",
    "required_sources": ["travel-policy-2026"]
}
```

For a classification system:

```
{
   "input": "I was charged twice for my subscription.",
   "expected_class": "billing_duplicate_charge"
}
```

For an agent:

```
{
    "input": "Find my last three invoices and summarize the largest charge.",
    "expected_tools": ["list_invoices", "get_invoice"]
}
```

The dataset defines the expected behavior of the system.

It becomes the equivalent of a regression-test suite for probabilistic software.

---

4. What Makes a Good Evaluation Dataset?

A dataset should not simply contain random examples.

It should represent the application's actual failure surface.

A useful dataset contains several categories.

Normal Cases

The common requests users make.

```
"What is our vacation policy?"
"Summarize this document."
"How do I reset my password?"
```

Boundary Cases

Inputs near decision boundaries.

```
"Does this qualify for reimbursement?"
```

where the answer depends on a subtle condition.

Difficult Cases

Questions requiring:

- multiple documents
- multiple reasoning steps
- long context
- ambiguous language
- conflicting evidence

Adversarial Cases

Inputs designed to expose vulnerabilities.

```
Ignore the previous instructions...
```

Negative Cases

Questions the system should refuse to answer or identify as unsupported.

```
"What was the CEO's private home address?"
```

Regression Cases

Previously observed failures.

Every important production failure should ideally become a permanent evaluation case.

This creates a powerful loop:

$$\text{Production Failure} \rightarrow \text{Evaluation Case} \rightarrow \text{Regression Protection}$$

The system becomes progressively harder to break in the same way twice.

---

5. Deterministic Tests

Not every AI behavior requires an LLM evaluator.

In fact, deterministic tests should be used whenever possible.

Suppose the model returns:

```
{
   "customer_id": "12345",
   "amount": 450.25,
   "currency": "USD"
}
```

You can test deterministically:

```
customer_id exists
amount >= 0
currency in allowed currencies
JSON schema valid
```

Likewise, if an agent is supposed to call:

```
get_customer()
```

you can verify:

```
tool_called == "get_customer"
```

and:

```
arguments.customer_id == expected_customer_id
```

There is no reason to ask another LLM to judge something that can be tested exactly.

This gives us an important principle:

Use deterministic evaluation wherever the behavior has a deterministic specification.

Use probabilistic evaluators only where necessary.

---

6. LLM-as-Judge

Many AI outputs cannot be evaluated with exact string comparison.

Consider:

Expected

> Revenue increased substantially in Q3, driven primarily by enterprise subscriptions.

Generated

> Enterprise subscriptions were the main driver of the strong Q3 revenue growth.

These strings differ substantially.

But they express essentially the same conclusion.

An LLM judge can evaluate properties such as:

- correctness
- relevance
- completeness
- style
- groundedness
- adherence to instructions

A conceptual evaluator might receive:

```
Question
Reference answer
Generated answer
Evidence
```

and return:

```
{
   "correct": 1,
   "grounded": 1,
   "complete": 0.8
}
```

LLM-as-judge is powerful.

It is not an oracle.

The judge itself is another probabilistic model and can introduce:

- bias
- inconsistency
- preference for verbose answers
- sensitivity to wording
- difficulty detecting subtle factual errors

Therefore:

> **An evaluator is itself a system that requires validation.**

---

7. Human Evaluation

Human evaluation remains important for difficult or high-value tasks.

Humans are particularly useful when:

- quality is subjective
- correctness requires domain expertise
- subtle reasoning matters
- evaluation criteria are difficult to formalize
- safety consequences are significant

A human evaluator might see:

```
Question:
Why did revenue decline in Q3?

Answer A:
Revenue declined because of weaker enterprise demand.

Answer B:
Revenue declined 7%, primarily because two large enterprise contracts
were delayed into Q4.
```

The evaluator may strongly prefer B because it is more specific and better supported by evidence.

Human evaluation provides high-quality judgments.

But it is expensive and slow.

This motivates a hierarchy:

```
Deterministic tests
        ↓
Automated metrics
        ↓
LLM evaluation
        ↓
Human evaluation
```

Use the cheapest reliable evaluation method available for each property.

---

8. Pairwise Evaluation

Sometimes asking:

> "Is this answer good?"

is difficult.

A simpler question is:

> "Which answer is better?"

Suppose we have:

```
Version A:
The policy allows business-class travel for international flights.

Version B:
The policy allows business-class travel for international flights
exceeding eight hours, according to Section 4.2.
```

A judge can compare them directly:

```
{
   "winner": "B",
   "reason": "B is more precise and cites the relevant condition."
}
```

Pairwise evaluation is useful for comparing:

- models
- prompts
- retrieval strategies
- chunking strategies
- agent policies
- context construction strategies

It also avoids some difficulties associated with absolute scoring.

Instead of asking for:

$$\text{Quality}(A) = 8.3$$

we ask:

$$A > B \ ?$$

This is often a more stable judgment.

---

9. Rubric-Based Evaluation

For more complex applications, define an explicit rubric.

For example:

Answer Quality Rubric

Dimension	0	1	2
Correctness	Incorrect	Partially correct	Correct
Groundedness	Unsupported	Partially supported	Fully supported
Completeness	Missing key facts	Some omissions	Complete
Relevance	Off-topic	Some irrelevant content	Direct
Citation	Missing	Partial	Complete

The evaluator can then produce:

```
{
   "correctness": 2,
   "groundedness": 2,
   "completeness": 1,
   "relevance": 2,
   "citation": 2
}
```

The advantage is that a single "quality score" is decomposed into actionable dimensions.

Instead of:

> "The model got worse."

you can discover:

> "Correctness remained stable, but completeness dropped 18%."

That tells the engineer where to investigate.

---

10. Accuracy

Accuracy is the most intuitive metric.

For classification:

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{total predictions}}$$

Suppose:

```
100 evaluation cases
93 correct
```

Then:

$$\text{Accuracy} = 93\%$$

Accuracy is excellent when classes and error costs are relatively balanced.

It becomes problematic when they are not.

Suppose:

```
990 normal cases
10 critical cases
```

A system that always predicts “normal” achieves:

$$\text{Accuracy} = 99\%$$

while completely failing the critical cases.

This is why evaluation metrics must match the application.

---

11. Precision and Recall

For retrieval and classification, precision and recall are often more informative.

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

Consider a security classifier.

A high-precision system generates few false alarms.

A high-recall system misses few true threats.

Which one is preferable depends on the application.

This illustrates a general principle:

> **There is no universally correct AI metric.**

Metrics encode the application's notion of failure.

---

12. Groundedness

For RAG systems, one of the most important properties is **groundedness**.

An answer is grounded if its claims are supported by the provided evidence.

Suppose the retrieved document says:

```
The reimbursement limit is $5,000.
```

and the model produces:

> The reimbursement limit is $5,000.

Grounded.

Now suppose it produces:

> The reimbursement limit is $5,000 and receipts must be submitted within 30 days.

If the evidence does not mention the 30-day requirement, the second claim is unsupported.

The answer may sound perfectly plausible.

It is not grounded.

A useful conceptual metric is:

$$\text{Groundedness} = \frac{\text{supported claims}}{\text{total factual claims}}$$

Groundedness is especially important because LLMs are optimized to produce plausible language, not necessarily evidentially constrained language.

---

13. Relevance

A response can be factually correct but still be a poor answer.

Question:

> "What is the maximum hotel reimbursement?"

Response:

> "The company has offices in Portland, Seattle, and Austin. The travel department was established in 2018. Hotel reimbursement is $350."

The answer contains the correct information.

It is not particularly relevant.

A relevance evaluator therefore asks:

> Does the answer directly address the user's request without unnecessary material?

This matters because increasing context and increasing answer length can sometimes produce more information while reducing usefulness.

---

14. Completeness

Completeness asks:

> Did the answer include all important information required by the question?

Consider:

> "What are the requirements for business-class reimbursement?"

Reference:

```
1. International flight
2. Duration greater than 8 hours
3. Employee must have director approval
```

Generated answer:

> Business class is available for international flights longer than eight hours.

The answer is partially correct.

But it omits the approval requirement.

Therefore:

$$\text{Correctness} = \text{high}$$

while:

$$\text{Completeness} = \text{low}$$

This distinction is important.

A system can be factually accurate about every statement it makes and still fail because it omits necessary information.

---

15. Hallucination Rate

Hallucination can be measured as unsupported factual content.

Suppose an answer contains ten factual claims.

Eight are supported.

Two are not.

Then a simple conceptual metric is:

$$\text{HallucinationRate} = \frac{2}{10} = 20\%$$

The exact implementation can be more sophisticated, but the key is to make hallucination measurable rather than treating it as a vague subjective property.

For high-stakes systems, it may be useful to track:

- unsupported claims
- contradicted claims
- fabricated citations
- unsupported numerical values
- unsupported entities

Numbers deserve particular attention.

LLMs can generate plausible-looking numbers that are completely absent from the evidence.

---

16. Tool-Call Success

Agentic systems introduce another class of metrics.

Suppose the model needs to:

```
1. Identify customer
2. Retrieve account
3. Calculate refund
4. Submit refund
```

The final answer might be correct-looking even if the agent failed to perform the required operation.

Therefore evaluate the trajectory.

For example:

$$\text{ToolSuccess} = \frac{\text{correct tool calls}}{\text{required tool calls}}$$

Also measure:

- correct tool selection
- correct arguments
- correct ordering
- unnecessary calls
- failed calls
- retries
- recovery behavior

For an agent, the final answer is only one part of correctness.

The **trajectory** matters.

---

17. Latency

Quality is not the only metric.

Suppose:

```
System A:
Accuracy = 91%
Latency = 2 seconds

System B:
Accuracy = 92%
Latency = 45 seconds
```

System B may be unusable for an interactive application.

Latency should therefore be measured across the pipeline:

```
Retrieval latency
Reranking latency
LLM latency
Tool latency
Total latency
```

Useful statistics include:

$$P50, \quad P90, \quad P95, \quad P99$$

Average latency alone can hide severe tail behavior.

For production systems, users experience the tail.

---

18. Cost

AI applications have an economic dimension.

A system might improve accuracy by 1% while doubling inference cost.

Whether that is a good trade depends on the application.

Track:

$$Cost_{\text{request}}$$

and:

$$Cost_{\text{successful task}}$$

The second metric can be particularly useful.

Suppose:

```
System A:
Cost/request = $0.02
Success rate = 80%

System B:
Cost/request = $0.04
Success rate = 95%
```

Then:

$$Cost_{\text{success,A}} = \frac{\$0.02}{0.80} = \$0.025$$

while:

$$Cost_{\text{success,B}} = \frac{\$0.04}{0.95} \approx \$0.042$$

System B is more capable but substantially more expensive per successful task.

Evaluation therefore becomes a multidimensional optimization problem.

---

19. The Evaluation Vector

Instead of thinking about application quality as one number, think of it as a vector:

$$\mathbf{Q} = (A, P, R, G, C, H, T, L, K)$$

where:

- A = accuracy
- P = precision
- R = recall
- G = groundedness
- C = completeness
- H = hallucination rate
- T = tool-call success
- L = latency
- K = cost

Different applications assign different importance to each dimension.

A search assistant may prioritize:

$$\text{Recall},\ \text{Precision},\ \text{Relevance}$$

A financial assistant may prioritize:

$$\text{Accuracy},\ \text{Groundedness},\ \text{Citation Quality}$$

An autonomous agent may prioritize:

$$\text{Task Success},\ \text{Tool Reliability},\ \text{Safety}$$

A consumer chatbot may prioritize:

$$\text{Helpfulness},\ \text{Latency},\ \text{Cost}$$

There is no universal leaderboard.

The evaluation suite must encode the actual requirements of the system.

---

20. Evaluation Is a Dataset Problem

An evaluation harness is only as good as its evaluation dataset.

Suppose you have:

```
1,000 easy questions
```

and:

```
0 difficult questions
```

Your system can achieve:

$$\text{Accuracy} = 99\%$$

while failing every important edge case.

The dataset therefore needs to represent the **distribution of real-world difficulty**.

A useful evaluation corpus might be divided into:

```
Common cases              50%
Difficult cases           20%
Multi-step cases          10%
Adversarial cases         10%
Boundary cases             5%
Historical regressions     5%
```

The exact distribution depends on the application.

The principle is:

> **Evaluate the cases where failure matters, not merely the cases that are easy to generate.**

---

21. Stratified Evaluation

Aggregate metrics can hide important failures.

Suppose overall accuracy is:

94%

Break it down:

```
Simple questions:        98%
Multi-document:          91%
Long-context:            83%
Adversarial:             72%
```

The aggregate number is now much less comforting.

This is why evaluations should be **stratified**.

Track metrics by:

- query type
- difficulty
- customer segment
- language
- document type
- tool usage
- context length
- retrieval depth
- model version

This produces a much more informative picture.

---

22. Regression Testing

Now comes the most important experiment.

Take your initial application:

```
Version 1
```

Run the evaluation suite:

```
Accuracy:        88.4%
Groundedness:    93.1%
Recall@10:       91.7%
Latency P95:     2.8s
Cost/request:    $0.018
```

Now change the system.

Perhaps you:

- switch embedding models
- change chunk size
- add reranking
- change the prompt
- switch LLMs
- alter context ordering

Run the exact same evaluation suite.

You might get:

```
Version 2
Accuracy:         90.1%
Groundedness:     94.0%
Recall@10:        95.2%
Latency P95:      5.4s
Cost/request:     $0.043
```

The new system is better on quality.

But it is also:

- almost twice as expensive
- nearly twice as slow

Whether Version 2 is better depends on the application's requirements.

This is precisely what evaluation should tell you.

---

23. Regression Reports

A useful regression report might look like:

```
                     V1        V2        Δ
-------------------------------------------------
Accuracy             88.4%     90.1%     +1.7%
Groundedness         93.1%     94.0%     +0.9%
Recall@10            91.7%     95.2%     +3.5%
Hallucination         4.2%      3.1%     -1.1%
Tool success         96.8%     97.1%     +0.3%
Latency P95           2.8s      5.4s     +2.6s
Cost/request       $0.018    $0.043    +139%
```

Now the engineering discussion becomes concrete.

Instead of:

> "The new system feels better."

you can say:

> "The new retriever improves recall by 3.5 percentage points and reduces hallucination by 1.1 points, at the cost of 139% higher inference cost and 2.6 seconds of P95 latency."

That is a meaningful engineering tradeoff.

---

24. Regression Gates

Once the evaluation harness exists, it can become part of CI/CD.

For example:

```
Pull Request
      ↓
Run evaluation suite
      ↓
Compare against baseline
      ↓
 +---------------+
 |  Regression?  |
 +------+--------+
          |
     +----+----+
     |         |
     ↓         ↓
   Yes        No
     ↓         ↓
  Reject    Merge
```

You might define policies such as:

```
Accuracy must not decrease > 1%
Groundedness must not decrease > 1%
Hallucination must not increase > 0.5%
P95 latency must not increase > 20%
```

These thresholds should be application-specific.

The important idea is that **AI changes can become testable deployment artifacts**.

---

25. But Don't Over-Automate Evaluation

There is a temptation to create one giant score:

$$\text{Score} = 0.3\,\text{Accuracy} + 0.2\,\text{Groundedness} + 0.2\,\text{Relevance} + 0.1\,\text{Latency} + 0.2\,\text{Cost}$$

This can be useful for ranking experiments.

It can also be dangerous.

A weighted score may hide catastrophic failures.

Suppose:

```
Accuracy        95%
Groundedness    95%
Latency         excellent
Cost            excellent
Safety          terrible
```

A weighted average could still look impressive.

For high-consequence properties, use **hard constraints** rather than averages.

For example:

$$\text{HallucinationRate} < 2\%$$

must be satisfied regardless of other improvements.

This is analogous to safety constraints in traditional engineering.

---

26. Evaluator Validation

LLM-as-judge creates a second-order evaluation problem:

> How do we know the evaluator is correct?

Suppose the judge scores:

```
Answer A = 9
Answer B = 6
```

But domain experts consistently prefer B.

The evaluation system itself is wrong.

Therefore validate the judge against a human-labeled sample.

For example:

```
100 examples
     ↓
Human evaluation
     ↓
LLM evaluation
     ↓
Compare judgments
```

Measure agreement.

If the evaluator systematically disagrees with experts, improve:

- the rubric
- the evaluator prompt
- the judge model
- the evidence supplied to the judge

or replace the evaluator.

This creates an important recursive principle:

> **The measurement system must itself be measured.**

---

27. Pairwise Evaluation for Model Selection

Suppose you are deciding between two models:

```
Model A
Model B
```

Run both on the same evaluation dataset.

For every example:

```
   Question
       ↓
  +----+----+
  |         |
  ↓         ↓
  A         B
  +----+----+
       |
       ↓
    Judge
       ↓
   Winner
```

Then calculate:

$$\text{WinRate}(A) = \frac{\text{A wins}}{\text{comparisons}}$$

This can be more informative than comparing independent numerical scores.

It is particularly useful for:

- model upgrades
- prompt variants
- retrieval algorithms
- context strategies
- tool-selection policies

Again, the key requirement is a stable evaluation set.

---

28. Production Evaluation

Offline evaluation is essential, but it cannot capture everything.

Real users produce unexpected inputs.

Therefore production systems should also collect evaluation signals.

A useful architecture is:

```
Production Request
        ↓
Application
        ↓
Response
        ↓
Telemetry
        ↓
Sampled Evaluation
        ↓
Failure Analysis
        ↓
New Golden Case
```

A production failure should ideally become a permanent offline test.

This creates a continuous learning loop:

$$\text{Production} \rightarrow \text{Failure} \rightarrow \text{Dataset} \rightarrow \text{Regression Test} \rightarrow \text{Improved System}$$

The evaluation dataset becomes a living representation of the system's known failure modes.

---

29. Evals Change How You Engineer

Without evaluations, an engineer tends to optimize through intuition:

```
Try prompt
    ↓
Looks better
    ↓
Try another prompt
    ↓
Looks better
    ↓
Ship
```

With evaluations:

```
Hypothesis
    ↓
Change
    ↓
Run benchmark
    ↓
Analyze metrics
    ↓
Inspect failures
    ↓
Accept / reject hypothesis
```

This changes the role of engineering judgment.

Intuition remains valuable for generating hypotheses.

But experiments determine whether those hypotheses are correct.

This is much closer to scientific experimentation than traditional feature development.

---

30. Evals as the Equivalent of Unit Tests

There is an important analogy with traditional software.

A unit test might say:

```
assert calculate_tax(100, 0.2) == 20
```

An AI evaluation might say:

```
Question:
What is the maximum reimbursement?

Expected properties:
- answer is $5,000
- claim is grounded in policy-17
- no unsupported conditions
- citation points to Section 4.2
```

The second test does not necessarily specify an exact output string.

It specifies a **behavioral contract**.

This is the right abstraction for probabilistic systems.

The model has freedom in how it satisfies the contract.

The evaluation determines whether the contract was satisfied.

---

31. Evals as the Missing Abstraction

Traditional software engineering has several layers:

```
Requirements
      ↓
Implementation
      ↓
Tests
      ↓
Deployment
      ↓
Monitoring
```

AI engineering needs a corresponding structure:

```
Behavioral requirements
         ↓
Context + model + tools
         ↓
Evaluation suite
         ↓
Deployment
         ↓
Production telemetry
         ↓
New evaluation cases
```

The evaluation suite becomes the bridge between **probabilistic behavior** and **engineering discipline**.

This is perhaps the most important conceptual shift of the week.

---

32. The Day 4 Exercise

Build a small evaluation harness around the RAG system from Day 3.

Start with perhaps 50–100 evaluation cases.

Each case should contain:

```
{
   "question": "...",
   "reference_answer": "...",
   "relevant_documents": ["doc-17", "doc-42"],
   "category": "multi_document"
}
```

Run:

```
Dataset
    ↓
RAG application
    ↓
Retrieved documents
    ↓
Generated answer
    ↓
Evaluator
    ↓
Metrics
```

Record at least:

```
retrieval recall
retrieval precision
answer correctness
groundedness
completeness
hallucination rate
latency
cost
```

Then deliberately modify the system.

For example:

Experiment 1

Change chunk size.

Experiment 2

Remove reranking.

Experiment 3

Change k.

Experiment 4

Add query expansion.

Experiment 5

Change the LLM.

Experiment 6

Change context ordering.

After each experiment, run the complete evaluation suite.

Do not rely on manual inspection.

Let the harness tell you what changed.

---

33. Deliberately Introduce a Regression

This is a particularly important exercise.

Make a change that you expect to make the system worse.

For example:

```
Baseline:
top_k = 5
```

Change:

```
top_k = 30
```

You may observe:

```
Recall@30        ↑
Precision@30     ↓
Context size     ↑
```

```
Latency          ↑
Groundedness     ↓
Answer quality   ↓
```

Now the evaluation suite has demonstrated something important:

optimizing an intermediate metric can make the end-to-end system worse.

This is a recurring pattern in AI engineering.

Local optimization does not necessarily produce global optimization.

---

34. Failure Analysis

When a test fails, preserve the entire trace.

A useful failure record contains:

```
Input
Model version
Prompt version
Retrieved documents
Retrieval scores
Context
Tool calls
Raw model output
Parsed output
Evaluator result
Latency
Cost
```

Then classify the failure.

For example:

```
RETRIEVAL_FAILURE
The required document was not retrieved.

CONTEXT_FAILURE
The document was retrieved but omitted from final context.

GENERATION_FAILURE
The evidence was present but the model produced an incorrect answer.

PARSING_FAILURE
The model produced correct information but invalid structure.
```

```
EVALUATION_FAILURE
The evaluator incorrectly classified a correct response.
```

This classification is extremely valuable.

It tells you which subsystem to change.

---

35. The Evaluation Matrix

A mature evaluation suite should not be a single list of questions.

Think of it as a matrix.

Dimension	Examples
Difficulty	easy, medium, hard
Retrieval	single-hop, multi-hop
Context	short, long
Evidence	consistent, conflicting
Time	current, outdated
Security	normal, adversarial
Output	concise, detailed
Tools	none, single, multi-step
Domain	finance, legal, technical
User intent	explicit, ambiguous

You can then ask:

> Where does the system fail?

rather than merely:

> What is its average score?

---

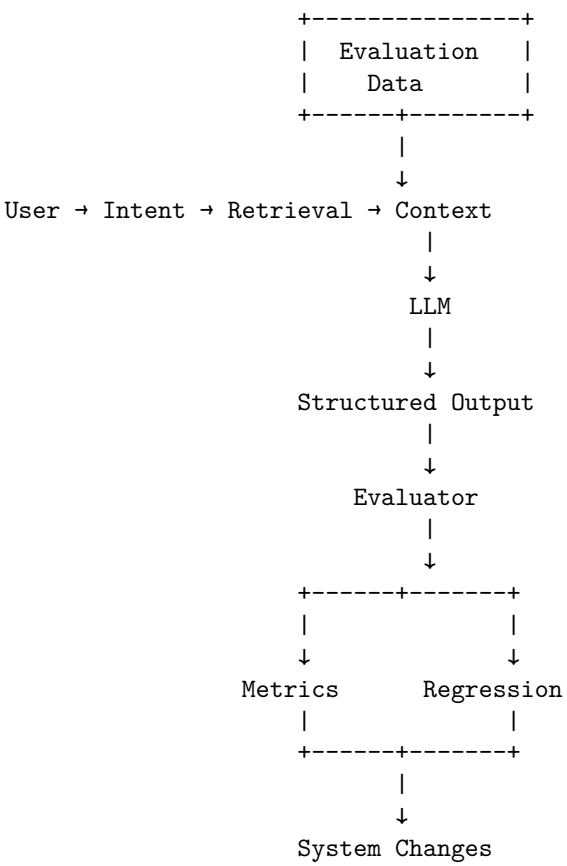
36. The Central Lesson

By Day 4, the architecture of the AI application should look very different from the simple model-centric picture we started with.

We began with:

```
User
  ↓
LLM
  ↓
Answer
```

We now have:



The model is now only one component inside a much larger engineering loop.

This is the point where AI engineering starts to look much more like software engineering.

---

37. The Deeper Principle

The traditional software mindset says:

> **If you cannot test it, you cannot reliably maintain it.**

For AI systems, we need a slightly different version:

> **If you cannot evaluate the behavior, you cannot reliably improve the system.**

The distinction between testing and evaluation reflects the probabilistic nature of the component.

Traditional tests often establish:

$$f(x) = y$$

AI evaluations often establish:

$$f(x) \in \mathcal{Y}_{\text{acceptable}}$$

where $\mathcal{Y}_{\text{acceptable}}$ is a set of outputs satisfying the behavioral requirements.

This is the appropriate abstraction for probabilistic software.

---

38. Day 4 Engineering Checklist

By the end of Day 4, you should understand:

- Why AI applications need dedicated evaluation systems
- How to construct a golden dataset
- How deterministic tests differ from probabilistic evaluation
- When to use exact-match evaluation
- When to use LLM-as-judge
- When human evaluation is necessary
- How pairwise evaluation works
- How to construct a rubric
- How to measure accuracy
- How to measure precision and recall
- How to measure groundedness
- How to measure relevance
- How to measure completeness
- How to measure hallucination
- How to evaluate tool calls
- Why latency and cost belong in the evaluation system
- Why aggregate scores can hide important failures
- How to stratify evaluation datasets
- How to build regression tests for AI behavior
- How to validate the evaluator itself
- How to turn production failures into permanent evaluation cases

Most importantly, you should be able to answer:

> **What does "good" mean for this AI application, and how do we measure whether a change made it better or worse?**

If the answer is subjective—

"It feels better"—

the system is not yet ready for serious engineering.

---

39. Key Takeaways

1. **Evals are the foundation of reliable AI engineering.** Without them, application development becomes guesswork.
2. **Golden datasets define behavioral expectations.** They are the AI equivalent of regression-test suites.
3. **Use deterministic tests whenever possible.** Do not use an LLM judge for properties that can be checked exactly.
4. **LLM-as-judge is useful but imperfect.** The evaluator itself must be validated.
5. **Human evaluation remains important.** Especially for subjective, complex, or high-consequence behavior.
6. **Pairwise evaluation is often easier than absolute scoring.** "Which is better?" can be more reliable than "How good is this?"
7. **Rubrics make evaluation actionable.** Correctness, groundedness, completeness, and relevance should be measured separately when they represent different failure modes.
8. **Metrics must match the application.** Accuracy is not sufficient for every system.
9. **Evaluate the entire system.** Retrieval quality, generation quality, tool behavior, latency, and cost all matter.
10. **Stratify evaluations.** A high aggregate score can hide catastrophic performance on difficult or important cases.
11. **Every production failure should become a regression test when practical.** This converts operational experience into permanent engineering knowledge.
12. **Evaluation should run continuously.** Model changes, prompt changes, retrieval changes, and context changes should all be measurable.

The central equation for Day 4 is therefore:

$$\boxed{\text{AI Engineering} = \text{Application} + \text{Evaluation} + \text{Feedback Loop}}$$

And the most important mental model is:

> **An AI application is not engineered when it works. It is engineered when you can measure its behavior, detect regressions, explain failures, and improve it systematically.**

That is the point where the probabilistic nature of LLMs stops being an excuse for unpredictability and becomes an engineering property that can be managed.